

Storing and Analyzing Large Graphs

What if all you need is a database

by

Puneet Mehrotra

M. Science, University of British Columbia, 2019

B. Engineering, Birla Institute of Technology and Science, 2013

A PROPOSAL SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR ADVANCEMENT
TO CANDIDACY FOR THE DEGREE OF

Doctor of Philosophy

in

THE FACULTY OF GRADUATE AND POSTDOCTORAL STUDIES
(Computer Science)

The University of British Columbia
(Vancouver)

March 2024

© Puneet Mehrotra, 2024

The following individuals certify that they have read, and recommend to the Faculty of Graduate and Postdoctoral Studies for acceptance, the candidacy proposal entitled entitled:

Storing and Analyzing Large Graphs

submitted by **Puneet Mehrotra** in partial fulfillment of the requirements for the degree of **Doctor of Philosophy in Computer Science**.

Examining Committee:

Margo Seltzer, Computer Science, UBC

Supervisor

Alexandra Fedorova, Electrical and Computer Engineering, UBC

Supervisory Committee Member

Thomas Pasquier, Computer Science, UBC

Supervisory Committee Member

Abstract

Graph-structured data is common, and data from many domains can be naturally expressed as graphs. As large graphs become ubiquitous, there is an increasing interest in storing them and processing them efficiently at scale. Graphs are rarely stored natively; most graph data is extracted from data resident in some persistent database and then transformed into an in-memory representation that can be used for graph-wide statistics. State-of-the-art graph processing systems are built around these extract-transform/preprocess-load (ETL) pipelines; this process must be repeated every time the data changes. Traditional relational databases have been avoided for running whole-graph analytics for many reasons ranging from an unintuitive interface to poor performance. Graph databases work well for storing and managing dynamic graph data; however, they are often overlooked as a candidate for long-running iterative graph algorithms. This is the central dichotomy of graph data management: graph databases allow for persistent storage of graphs in a native representation, but they are outperformed by specialized graph processing systems that require a costly ingest and preprocessing phase before executing efficient graph analytics queries.

This thesis claims that it is possible to store and manage dynamic graph-structured data in a performant and extensible way. We designed and implemented a transactional graph database system based on a production-ready general-purpose key-value storage engine, providing an API that lets users interact with their data efficiently. Our system is extensible and allows the user to store multi-layer property graphs and gives them a choice of three different storage formats. The API is flexible enough that users can express their graph computations in terms of vertex or edge operations; the system itself does not restrict the paradigms that can be used. Our system is designed for a single-node, single-socket environment, and we show that high performance on graph analytics tasks can be achieved without scaling to a cluster. Additionally, we show that our system scales to large graphs that scale to billions of edges and can handle transactional and analytical workloads simultaneously.

Our architecture provides the best of both worlds: the data management capabilities one expects from a database, with the performance of a specialized graph processing system with none of the ETL or preprocessing costs involved in those solutions.

Lay Summary

A lot of scientific and real-world data encodes relationships between entities and is often expressed as a graph. As we produce ever larger graph datasets, there is a growing need to efficiently store and process them. Traditionally, graph data is extracted from another data source (such as a database), transformed into a usable format for analysis, and loaded for analytics; this process must be repeated every time the data changes. This thesis presents an intervention in this paradigm of extract-transform/preprocess-load by presenting a system to store the graph in an analytics-friendly format. This approach reduces the time and resources needed for the transformation and preprocessing steps, ultimately lowering the cost of running analytics.

Chapter 1

Introduction

PM ▶ *How and where should I point out that Graph data management systems include graph databases and graph processing systems? I need to make this distinction to refer to both GDBs and GPSs collectively.* ◀

TLS ▶ *New data are being generated rapidly, and there is an ever-present and growing need to store and analyze them efficiently.* ◀ Much of this data has some inherent schema or structure, and relational databases are a natural choice for efficient storage. Relational databases (RDBMS) have been the backbone of data management for six decades and have been repeatedly extended to service new and emerging use cases [20, 42, 73, 74, 83]. RDBMSs are well-suited for storing and querying structured or semi-structured data on workloads with high volumes of small updates and stringent consistency requirements; these tasks are called online transactional processing (OLTP) workloads. Larger scale data analysis frequently requires full dataset scans and extensive join processing, and these data warehouse or business intelligence (BI) workloads are called Online Analytical Processing (OLAP). Several relational OLAP databases and data warehousing solutions exist to enable such enterprise scale workloads [9, 20, 34, 63].

However, in some applications, the relationships captured in the data are themselves a fundamentally important object of study. In myriad applications ranging from social network analysis to brain imaging, these relationships form a graph, and that graph itself is important to study and understand. For example, a social network maintains information about every user in the network, such as their name, occupation, address, interests, etc. Meta (Facebook) uses a relational database to store this social network database [22]. However, the "network" part of a social network represents important relationships such as "follows" or "likes". These relationships form a large graph. Each vertex of this graph ~~would store properties for each user~~ (i.e. schema from the relational table for users) **while** each edge ~~would have a type (or label) defining which relationship is being captured~~. Graphs in which the nodes have properties and the edges have labels are called Labeled Property Graphs [18].

Graph analysis tasks typically access the whole (or a significant part of the) graph. If we store the graph **purely** in its relational form, **relations are** encoded as joins, and graph traversals turn into recursive self-joins [10]. As a result, RDBMSs do not perform well for graph analytics tasks, as computing these recursive self-joins is expensive [19].

In response to this mismatch between graph analysis tasks and relational databases, we have witnessed an explosion in special-purpose systems for storing and processing graph-structured data. Specialized graph databases such as Neo4j [8] and ArangoDB [5] follow ~~very much~~ in the tradition of relational databases, providing a schema-oriented organization and a high-level query language. These systems work well for OLTP workloads but frequently exhibit **less-than-stellar** performance on OLAP tasks. In contrast, a second class of graph processing systems, such as Pregel [53], GraphX [33], PowerLyra [25], GraphChi [45] and Ligra [70] are specifically designed to provide outstanding OLAP performance but may not provide OLTP-style query capabilities.

OLTP queries on graphs are similar to OLTP queries on more conventional data; for example, in a social network,

a user might wish to see the latest posts made by their friends or the company for which their friend works. Such interactive queries are latency-sensitive, with many tasks, such as fraud detection in financial networks, demanding 20 ms latencies [62].

Analytical workloads on graphs (Graph OLAP) are queries about the network that can be answered only through a "whole-graph" analysis. For example, a cloud service provider might wish to establish which clients have been exposed to an attack by using the login information log. One possible way to do this would be to run traversal queries from the affected clients and establish a *blast radius* of possibly infected machines. This task must interact with large parts of the graph, if not the whole graph, and will typically take longer to finish.

TLS ▶ This thesis presents the design of a scalable and efficient hybrid "best of both worlds" graph data management system that stores graphs in a database structure but can handle both analytics tasks and transactional queries using analytics-friendly formats. ◀ Designing such a system needs close examination of the state-of-the-art to identify extant designs that are aligned with our design goals (Section 1.1). Additionally, we identify a few anti-patterns that we wish to avoid (Section 1.2) while designing our hybrid system. Together with our novel contributions in graph state management, principled persistent layout design, and thread scheduling, Flexograph can achieve performance comparable to in-memory graph processing systems while also being competitive with graph databases.

1.1 State of the Art: What's good?

State-of-the-art systems typically assume that graph databases and graph processing systems typically run on a distributed collection of nodes. We start with the assumption that the best distributed system is built from the best single-node system. Based on this assumption, we identify a set of requirements for building such a single-node graph processing system.

1. **Efficient out-of-the-box algorithms and a rich, useful API for expressing computations:** Using efficient algorithms is essential for fairly evaluating the system and establishing areas that need improvement [17, 56, 57]. Additionally, it is useful to provide an intuitive and flexible API that allows users to write custom algorithms.
2. **Using well-designed data structures that can be parallelized without incurring severe penalties from locking overheads** is critical in designing parallel graph processing systems that leverage today's multicore compute infrastructure and fast storage devices.
3. **Using data layouts that maximize sequential access to the graph are preferred** and painstakingly designed. This has been the central theme for much research in semi-external graph processing¹, and every system has its clever scheme to address this problem for its API and computation model. Creating specialized data layouts requires preprocessing the graph, which is a significant overhead relative to the cost of running many computations. Preprocessing costs are rarely amortized.

We build upon these ideas to create Flexograph.

To eliminate the overhead of compare-and-swap or locking-based update propagation, we use a lock-free implementation for key datastructures in our code. **PM** ▶ I want to revisit the blaze-style non-blocking propagation that uses two buffers for each thread. Jack prototyped it, but I don't recall why we never used it. ◀

¹In semi-external systems, vertex, and algorithmic state is maintained in RAM and the graph edges are read from the storage device

Flexograph prototypes three different data layouts that each have their own benefits and drawbacks, which we analyze in [PM ▶ Add forward section reference. ◀](#). We provide a detailed comparative analysis of these data layouts and hints for choosing a data layout based on the read-write mix an evolving graph might experience.

Contrary to the popular practice of building a custom storage engine for accessing the graph, we show that building a graph database upon a key-value store provides performance comparable to other persistent graph processing systems. And, when we leverage the key-value store cache, we provide in-memory performance comparable to that of main-memory graph processing systems.

1.2 State of the Art: What needs to be changed?

Graph data management has a fork in the road: one set of systems focuses on high-throughput whole-graph analytics, while the other focuses on low-latency OLTP-style processing; data is transferred between these systems via extract-transform-load pipelines. Flexograph is designed to find a middle ground in this landscape and reduce cross-silo data movement and preprocessing costs. There are a few specific issues with the state-of-the-art graph processing systems we address with Flexograph, namely, single-node optimization, in-database analytics, and bridging the gap between graph OLAP and graph OLTP systems.

1.2.1 Single-node optimization before scale-out

TLS ▶ *Past research has conclusively shown that premature distribution of tasks based on poor single-node baselines is doomed to perform badly.* ◀ McSherry et al. [56] demonstrated that several popular distributed GPS evaluated their scalability by comparing, not against an efficient single-threaded implementation, but by only showing how the run time changes as the total number of threads increases. When McSherry compared these scalable systems to an efficient sequential implementation, he revealed that some systems could not match “laptop performance” regardless of how many machines were used! This stunning result led to several attempts at standardization of benchmarking practices for GPS [6, 16, 23, 57]. Each proposal identified the need to use more performant algorithms and report more comprehensive performance metrics. Considering that distributed graph processing is inefficient due to the communication costs, the main benefit of using a cluster for graph processing is that the aggregate RAM might be sufficiently large to enable processing graphs entirely in memory, even if their size is larger than the memory of a single machine. The Waterloo survey on the state of graph processing reveals that large graphs are ubiquitous, and several of their respondents reported operating on uncompressed graphs larger than 1 TB; the exact upper limit on the size of these graphs is unknown [67, Section 3.2]. While servers with more than a few terabytes of RAM exist, they are expensive and, as we and others have shown, unnecessary. Scale-out systems built from semi-external graph processing systems should provide better price performance than a distributed graph/data processing system. We want to design and optimize a single-node, single-socket graph database before requiring distribution.

1.2.2 Best-of-both worlds design for mixed OLAP and OLTP workloads

TLS ▶ *As reported in section 2.3, many graph processing systems require expensive pre-processing steps to construct a suitable on-disk data layout that allows fast sequential access to the graph.* ◀ Time spent preprocessing the graph is rarely amortized; it varies by the freshness requirements for the workload and the rate at which new snapshots or increments need to be analyzed.

A better alternative is to natively store the graph in a format similar to or close to the in-memory representation required for efficient access. Graph databases do this but are optimized for transactional point queries rather than

full-graph analytics. Neo4j [8], for instance, stores all graph data on disk as "linked lists of fixed size records" [59]. A linked-list representation of graphs is great for connectivity queries but is bound to cause catastrophic random access overhead when running whole graph analytics. Newer research systems such as GraphFlowDB [35] provide a more methodical analysis of design decisions required to store a property graph in memory. Still, they do not persist or evaluate the system on whole-graph analytics benchmarks. Storing a graph larger than available RAM requires careful persistent and in-memory data layout co-design, which performs well on both OLAP and OLTP-style workloads.

1.2.3 In-Database Analytics

We examined the use of specialized graph processing systems for analytics tasks and know they are typically not transactional and do not support point queries. Graph databases are transactional and perform well on point queries but are orders of magnitude slower than specialized graph processing systems [61] on whole-graph analytics tasks. Therefore, state-of-the-art graph data management has evolved around a split system design: graph database systems for point queries and specialized high throughput graph processing systems for whole-graph analysis tasks. This leads to a freshness mismatch between the data stored in the original data store, what's available for point-query analysis following ingestion into a graph database, and what's available for batch analysis in a specialized graph processing system. The split system design also causes the creation of data silos which need constant ETL and data movement to maintain consistency.

These problems have an equivalent in the relational database world: the constant data transformation needed to maintain consistency between a row-store and a column store [15], and the subsequent development of Hybrid Transactional and Analytical Processing (HTAP) databases is a promising solution. Similar solutions must also be developed for analyzing graphs, and recent commercial graph databases, such as GART [68], have started offering solutions for real-time graph analytics workloads. GART stores the extracted graph in memory since it has been designed for latency and freshness-sensitive tasks, such as fraud detection, and not for further downstream batch processing tasks we target. Graph HTAP systems should be designed to handle efficient real-time analytics and more intensive whole-graph processing tasks.

1.3 What have we done?

TLS ► This thesis presents the design of a flexible and performant hybrid graph analytics and processing (HGTAPE) database system that performs better than custom-created graph processing systems on analytics tasks and is competitive against graph databases on transactional point queries. ◀

Our system, Flexograph, has been designed for a single-node setting and provides a fully transactional API for updating and querying a graph. We provide snapshot-level read-only iterators for fast serial reads for analytical tasks. Flexograph has been designed to eliminate any preprocessing tasks needed before running analytics and provides two layouts we designed to get strong sequential iterator-based read performance common in analytics tasks. The API is flexible enough to let users write algorithms as vertex or edge programs; we demonstrate our performance by modifying GAP-BS to use our API and demonstrating that our analytics tasks have robust performance compared to other semi-external and in-memory systems when factoring in their pre-processing times.

Our system handles property graphs, and we evaluate the benefits of storing the graph's structure independently from their properties and contrast it with the choice to store them together. **PM** ► My idea: structural graph separate from property store. Jack's idea: storing them together. ◀

The contributions of this thesis are:

1. The design of a Graph HTAP database takes a "best-of-both-worlds" approach to deliver robust performance on whole-graph analytics tasks and transactional queries without the need for constant preprocessing or ETL.
2. We present a principled analysis of analytics friendly data layouts and how we use a mature key-value store (WiredTiger [4]) as the storage engine to store and access the graph. The API provided by Flexograph allows the user to do fast neighborhood scans by using scan cursors and provides an API for persisting the algorithmic state of graph processing tasks to bootstrap future rounds of computations.
3. Flexograph provides a multi-threaded bulk-insertion tool for quickly loading the initial graph into the database. Most other bulk-insertion tools require a single-writer to the database due to consistency requirements; we relax these requirements for fast bulk insertion.
4. We provide a comprehensive performance analysis of Flexograph against graph databases and graph processing systems (in-memory, semi-external, and dynamic) to show that we are competitive in all cases, but with lower data migration cost due to low/no preprocessing and ETL costs.

The remainder of this thesis proceeds as follows: In Chapter 2, we provide the background that motivates our work. We begin by describing the data structures used to represent graphs efficiently in memory using layouts that maximize sequential access, particularly the most popular layouts: Compressed Sparse Row (CSR) format (§2.3.3), Adjacency Lists (§2.3.2), and Edge Logs (§2.5.2). We discuss the design of data layouts in graph processing systems in Section §2.4 and graph databases in Section 2.6. Following that, we examine how semi-external systems create on-disk layouts that allow efficient access to the graph. We then turn our focus towards graph databases, and examine the transactional queries for which they are optimized. With this context in place, we contrast graph databases and graph processing systems, establishing the central dichotomy of graph data management and lay the foundation to discuss our hybrid graph transactional and analytical system design.

Chapter 3 presents the design of Flexograph. We begin by describing our graph API and then introduce the three layouts for which we provide that API. Next, we describe the iterators we provide for efficiently accessing graph elements: nodes, edges, and neighborhoods of nodes. We conclude with a description of work assignment and thread scheduling for reduced-conflict writes to the graph. Finally, we discuss the optimizations we provide for maintaining the algorithmic state and using results from previous computations to jumpstart new rounds of computation.

Chapter 4 describes how we extend Flexograph to store property graphs [12]. We begin with a description of the design alternatives available to us for storing properties with graph objects **PM** ► *inline with structure or independently* ◀ and describe the cost of using these two approaches. We compare these two designs with each other and against other dynamic graph processing systems and graph databases on LDBC queries, and show that we can perform similar to our competitors.

We discuss future work and conclude in Chapter 5. **PM** ► *SO much future work, need to think what will be useful based on how the previous section turns out.* ◀

Chapter 2

Background

TLS ▶ Graph data management systems can be characterized along four axes: the types of graphs they store and operate on, whether these graphs change over time, the types of algorithms they support (also known as the programming model), and the types of workloads for which they are designed. ◀ Flexograph sits at the intersection of graph databases and graph processing systems, making it essential to understand their design, supported workloads, and their similarities and differences.

First, we ~~will~~ formally define graphs and discuss the Labeled Property Graph model, a standard formulation that guides the design of most graph data stores. With this foundation, we ~~will~~ examine how individuals utilize and interact with their graph data. Specifically, we ~~will~~ explore access patterns corresponding to the workloads a system supports. We ~~will~~ delve into both graph OLTP and ~~graph~~ OLAP workloads, analyzing the access patterns they generate and the real-world applications ~~they correspond to~~.

This context sets the stage for discussing the data structures used to store graphs in memory and the challenges faced when the graph is dynamic or too large to fit in RAM. Next, we ~~look into~~ graph processing systems, reviewing their design considerations for storing graphs on disk and the data layouts necessary for fast sequential access to subgraphs as required by the workloads.

Finally, we ~~will~~ briefly ~~overview~~ graph databases, highlighting the data layouts used by popular commercial graph databases and their optimizations for fast seeks and scans within graph neighborhoods. This comprehensive review ~~will allow~~ us to understand the field of graph data management, and place the Flexograph design ~~into~~ perspective.

2.1 Graph Models

TLS ▶ A graph G is defined as a pair $G = (V, E)$ of sets V and E such that $E \subseteq V \times V$; V is the set of vertices (also called "nodes") of the graph G , and E is the set of edges of G [30]. ◀ The vertices are the entities modeled by a graph, and an edge represents a ~~relation~~ between the vertices to which it connects. The edges of a graph can be directed or undirected. In a directed graph G , an edge $e = (u, v) \in E$ is an ordered pair, where u is called the "source" vertex and v is the "destination" vertex. ~~On the other hand~~, if G is undirected, then an edge $e = (u, v) \in E$ is a ~~set~~ of two vertices. A weighted graph has a "weight" associated with each edge. Formally, a weighted graph G is modeled as a triple (V, E, w) , where $w : E \rightarrow \mathbb{R}$ maps edges to weights.

TLS ▶ While this formulation is suitable for graph theoretic studies, real-world graphs have properties and labels associated with vertices and edges, and our simple formulation needs ~~enhancement~~ to cover these extra features. ◀ The extended graph model that satisfies the requirement to have properties and labels associated with graph elements is called the

Labeled Property Graph (or simply a Property Graph) Model [13]. We use the term Property Graphs (PG) to refer to these graphs in the remainder of this thesis. Each vertex and edge in a PG has labels used to construct subsets of vertices and edges. For instance, in a social network, the nodes could be labeled "users" or "organizations," and the relationship labels could be "is friends with" or "follows." Additionally, the nodes and the edges in a PG could have an arbitrary number of *properties* (also called *attributes*) associated with them. These properties are modeled as (*key*, *value*) pairs, where *key* identifies the property and the *value* is the corresponding value of that property. A more formal definition of Property Graphs can be found in the survey papers by Angles et al. [13] and Yakovets et al. [21].

However, before discussing workloads, it is essential to understand that most graph databases offer some subset of the Property Graph Model. Neo4j [8] allows the vertices in the graph to have an arbitrary number of labels, but it allows only one edge label per edge. ArangoDB [5] allows only one label per vertex or edge in the graph.

2.2 Graph Workloads

In Chapter 1, we briefly explored typical relational database workloads: Online Transactional Processing (OLTP) and Online Analytical Processing (OLAP). OLTP workloads involve tasks that insert or update data in a database at high volumes using transactions. These systems are optimized for write operations, are latency-sensitive, and aim for short transactions that modify only a small portion of the database. In contrast, OLAP workloads are read-heavy and are used to analyze and aggregate data. While analytical processing tasks also involve high volumes, they are generally less sensitive to latency and are designed to maximize the throughput of data access.

Similarly, graph data management systems can handle both OLTP and OLAP workloads. An example of a graph OLTP workload is a neighborhood query, such as finding a user's friends in a social network graph. These queries tend to access small parts of the graph, are short-running, and transactional. Other graph OLTP tasks include inserting or updating vertices or edges, looking up nodes or edges, and other interactive queries. These typically involve immediate responses and real-time data manipulation.

In contrast, graph OLAP queries are usually long-running and complex tasks that often involve reading large portions of the graph, if not the entirety. Familiar examples of analytics workloads include running Breadth-First Search (BFS), PageRank, community detection tasks (such as triangle counting and label propagation), shortest path computations (SSSP), pattern mining, and many others.

To better understand the graph workloads and how they access the graph, it is helpful to classify them based on how much of the graph they access (also called the scope of the query). A recent survey by Besta et al. [18, Section 4.2.1] provides a handy classification of graph queries on graph databases. We briefly summarize their classification here to complete our description of the research landscape.

2.2.1 Types of Graph Queries by Scope

~~We now examine the types of graph queries common on property graphs.~~

1. **Local Queries:** Local queries on a property graph (also called point queries) involve accessing a single vertex or edge in the graph, typically to set or retrieve some associated property or to filter edges/nodes with a certain label.
2. **Neighborhood Queries:** Neighborhood queries are used for finding the edges incident to or emanating from a vertex. This query is used to find the reachable nodes from a node (also called a neighborhood). An example of a neighborhood query is to find a user's friends in a graph.

3. **Traversals:** Traversals on a graph involve exploring the graph layer by layer from a particular "root" node. Starting from a given source vertex, a traversal query visits all the neighbors of the root vertex first before moving on to the neighbors of those neighbors, and so on. This process continues until all reachable vertices have been visited. Often, such queries are restricted to explore nodes or edges of a particular label or having a specific desired property. An example of a traversal query is performing breadth-first search (BFS) or depth-first search (DFS) on the graph.

Graph databases support these queries, though their performance on these tasks varies quite a lot, as we shall see in Section [PM ▶ add forward reference to GraphDB performance.◀](#). Static graph processing systems are specialized to handle whole-graph OLAP queries.

Wherever there are workloads of broad commercial interest, a benchmarking council is looking to formalize a specification; graph data management systems are no exception. The leading benchmarking association for graph workloads is the Linked Data Benchmark Council (LDBC) [1], and they have published several benchmark specifications to standardize the benchmarking practices on graph data. LDBC provides two popular benchmarking suites: the Social Network Benchmark (SNB) [3] and Graphalytics [2].

LDBC-SNB defines two subclasses of transactional workloads: interactive and business intelligence workloads. The interactive workload is centered around "complex read queries that access the neighborhoods of a given node and update operations that continuously insert new data into the graph." [3]. Besta et al. [18] provide a breakdown of the interactive workload into short read-only queries (neighborhood lookup for a node or short traversals), complex read-only queries (traverse a more significant part of the graph and used to assess query optimizers), and transactional update queries (insert/modify/remove a node and its incoming or outgoing edges). The Social Network Benchmark also provides the LDBC Business Intelligence workloads, which gauge the system's performance under more expansive reads that fetch large parts of the graph. Typical queries include tasks that sort, order, aggregate, and derive maximum or minimum values from the data.

2.2.2 Whole-Graph Analytics

Whole-graph analytics tasks involve computations that necessitate accessing the entire graph to determine specific properties, such as vertex centralities, shortest paths, or clustering coefficients. These tasks are essential for comprehending the global structure and dynamics of the graph. For instance, vertex centrality measures help identify the network's most important or influential nodes. Similarly, community detection algorithms analyze the graph to find clusters or groups of nodes more densely connected than the rest of the graph. Community detection is instrumental in social network analysis, where identifying groups of closely connected individuals can provide insights into social structures and interactions. Whole-graph analytics are crucial for applications requiring a comprehensive understanding of the network's topology and can inform decisions based on the overall connectivity and relationships within the graph.

These analytics tasks involve repeatedly accessing the neighborhoods of every node and updating some global algorithmic state, resulting in random access to vertex data and random writes when updating the algorithmic state. Graph OLAP or graph processing systems are designed to handle this class of analytics tasks. The LDBC Graphalytics benchmark and the GAP Benchmark describe the most important classes of graph analytics kernels based on their frequency of use in applications and their diverse computational complexities.

Graphalytics includes six kernels: Breadth-First Search (BFS), PageRank (PR), Weakly Connected Components (WCC), Label Propagation for Community Detection (CDLP), Clustering Coefficient, and Single-Source Shortest Path (SSSP). GAP includes BFS, PR, Triangle Counting (TC), Betweenness Centrality (BC), and Connected Components (CC).

GraphMineSuite distinguishes itself by noting that most graph kernels used for graph mining are significantly more computation-heavy than the "low-complexity" kernels used in most applications and benchmarking suites such as GAP and Graphalytics. GraphMineSuite provides a framework for evaluating and a reference implementation for graph pattern matching, graph learning algorithms, optimization problems (such as minimum spanning tree and minimum cut), and vertex ordering tasks."

2.3 Graph Representations

In §2.1, we examined the formal definition of graphs. This section describes how graphs are represented by describing the common data structures employed for storing them. We begin with the most common way to represent graphs and then follow up with other, more compact layouts designed to enable sequential access to graph neighborhoods.

2.3.1 Adjacency Matrix

The canonical representation of a graph uses an $n \times n$ matrix A , where n is the number of vertices in the graph; an element $a_{ij} = 1$ in A indicates the presence of an edge between vertices i and j . Depending on the matrix's definition, edges can be directional or non-directional (we use the term *undirected graphs* to refer to these). Hence, the edge a_{ij} can either indicate a directed edge $i \rightarrow j$ or an undirected edge $i \leftrightarrow j$. It is common to represent undirected edges between vertices i and j as two directed edges $i \rightarrow j$ and $j \rightarrow i$.

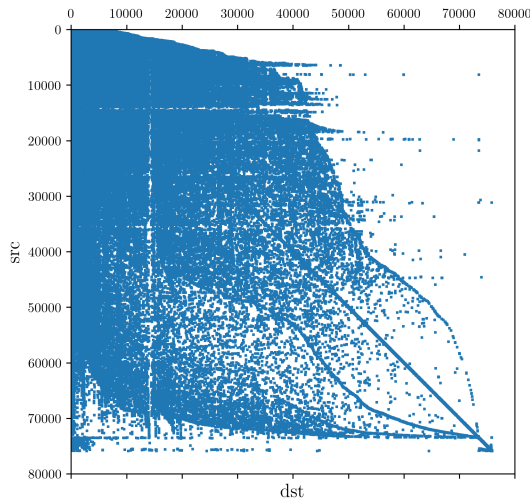
TLS ▶ Real-world graphs tend to follow the power law distribution, have small diameters, and exhibit communities [24, Section 2].◀ A power-law distribution of degrees leads to a few *hub* nodes with very high degrees and many nodes with very low degrees (heavy tail of degree distribution). A direct consequence of the power law graph is that the resulting adjacency matrices are sparse and, therefore, memory inefficient. Figure 2.1 shows the adjacency matrix and the degree distribution plots for soc-epinions1 [71], a small network with around 76000 nodes. The white space in the adjacency matrix visualization denotes the sparsity pattern, which translates to empty cells in an $n \times n$ two-dimensional array representation of the graph. This wasted space in an adjacency matrix leads to poor cache utilization, which is already challenging due to the random access pattern inherent in graph traversal. Better data structures that are more compressed than an adjacency matrix are essential to minimize the space requirement and maximize cache utilization.

We now examine a few memory-efficient data structures commonly used in graph processing systems and the cost of creating, updating, and finding neighborhoods of a node in each representation. For the remainder of this discussion on graph data structures, we will use the sample graph shown in Figure 2.2a.

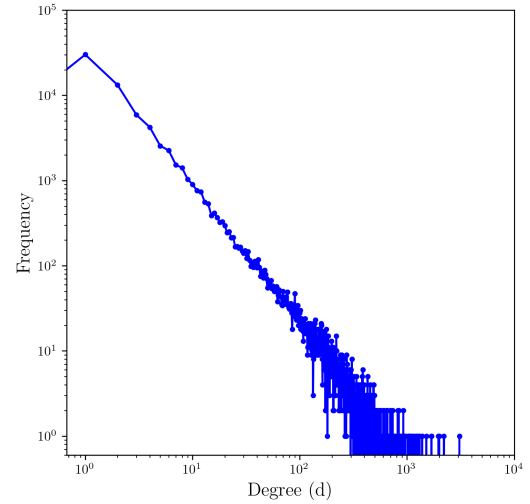
2.3.2 Adjacency Lists

TLS ▶ Adjacency lists are a natural evolution from adjacency matrices and are constructed by breaking down the graph into per-vertex neighborhood lists.◀ For instance, consider the adjacency matrix shown in Figure 2.2b. To construct the neighborhood list for each node, we construct a vector containing the column index j where $a_{ij} = 1$ in the adjacency matrix. The corresponding adjacency list representation of the graph is shown in Figure 2.2c. The memory requirement for the adjacency list representation is $O(|V| + |E|)$, which is superior to the $O(V^2)$ required for an adjacency matrix.

TLS ▶ Adjacency lists allow sequential access to a node's neighborhood. However, a random seek is required to access the neighborhood (the vertex contains a pointer to the list).◀ Adjacency lists are dynamic data structures. Inserting new edges into an unsorted adjacency list representation has an amortized $O(1)$ cost; however, if the list must be kept sorted, the cost increases to $O(\log(n))$. Typically, two adjacency lists are maintained for a graph: one for the



(a) The adjacency matrix for the network



(b) Degree distribution plot for the network

Figure 2.1: soc-Epinions1 is a sparse real-world graph that follows a power-law degree distribution. The adjacency matrix is memory inefficient. The power-law distribution shows that a few vertices have a high degree (hub nodes), while most vertices have a low degree.

in-neighborhood and another for the out-neighborhood.

2.3.3 Compressed Sparse Row

TLS ▶ *Compressed Sparse Row(CSR) [66] is a very compact, immutable data structure that uses two flat arrays to store the neighborhood lists for nodes in a graph.* ◀ Assuming a non-symmetric sparse matrix A , we create two integer vectors: *vertices* and *edges*. For each $v_i \in V$, $vertices[v_i] = idx_{v_i}$; where idx_{v_i} is the index in *edges* where v_i 's adjacency list begins. The contents of $edges[idx_{v_i}]$ to $edges[idx_{v_i+1}]$ are the sets of vertices v_j such that $a_{ij} \neq 0$. CSR stores a graph $G = (V, E)$ in $O(|V| + |E|)$, but since it is immutable, **it needs reconstruction upon any changes to the graph**. The edge array might need reallocation to accommodate an incoming edge, and the old edges must be moved and shifted around the new edge. This is a pretty serious limitation for evolving graphs, and much research has been done to create resizable CSRs for graphs.

Common variations of CSRs that are insertion-friendly include:

1. **In-place Updates:** CSRs can be created to allow in-place insertion and deletion of graph elements without needing a full, expensive rebuilding of the data structure. NetworKit[72] doubles the size of the initial array and reserves space for new edges. A variant of the CSR called the Packed CSR [80] leaves space between elements, allowing for faster insertion and deletion of graph elements at the cost of a slowdown in traversals and an increase in the space overhead.
2. **Batching updates:** Many systems batch updates and preprocess the updates to make the insertions faster. Systems such as the one described by Madduri et al. [52] batch updates, order them by vertex IDs and sort them before applying the updates.
3. **Multi-versioning and Delta Updates:** An alternative design to support fast inserts into a graph is to store all updates in a separate structure called a Delta Map. This use of delta maps allows for running graph

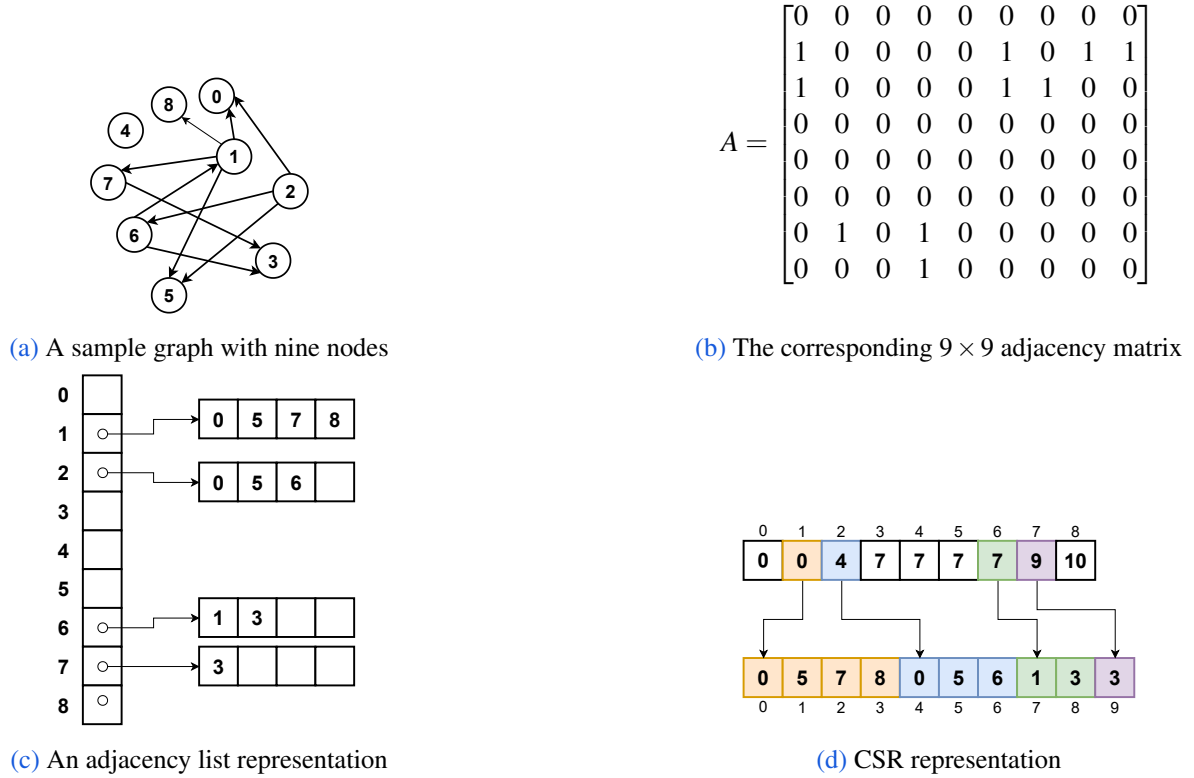


Figure 2.2: A small directed graph with nine nodes and ten edges and the different representations used to store it

analytics on snapshots of the graph at different times in its evolution. LLAMA[51] is one such snapshot-based graph system that supports multi-versioning and stores the delta updates as separate concurrently accessible snapshots. There are some downsides to using a snapshot-based system for analytics. A high ingestion rate leads to many delta snapshots since the updates get fragmented over multiple snapshots. An analytics job must read many snapshots, and this causes significant performance degradation. **It is common to merge and consolidate updates into the CSR; this process is called compaction.** It is similar to garbage collection and leads to increased latency.

2.3.4 Tree-based data structures and layouts

Some graph processing systems use tree-based data structures to store evolving graphs. The system might use a **B⁺ tree** or a **Log-Structured Merge Tree (LSM Tree)** to store the graph, and the choice is made based on the system's design goals. Relational databases have long stored data using B⁺ trees, and more recent systems such as RocksDB use **LSM Trees** ~~to store key-value pairs~~. While both LSM and B⁺ trees support concurrent read-write operations, B⁺ trees are typically used when the system needs a high read bandwidth, while LSM Trees are used when the system needs to support high-volume writes. While B⁺ trees and LSM trees are more commonly used to store data on disk, some systems use tree-based data structures to store graphs in memory; we examine one such system.

Teseo [29] is a dynamic graph processing system that uses an B⁺ Tree derivative, called a *fat tree* to store the

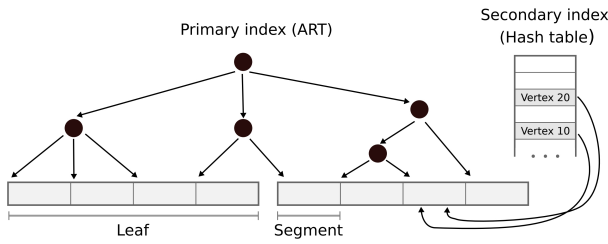


Figure 2.3: The Fat Tree layout used by Teseo.

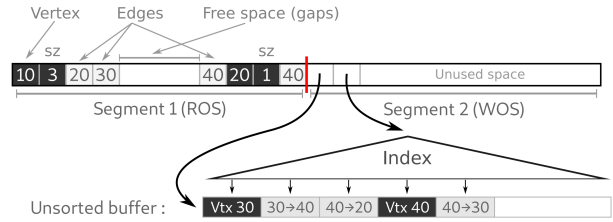


Figure 2.4: The segment structure of a leaf node in Teseo.

Figure 2.5: The data layout used by Teseo to store evolving graphs in memory. Taken from de Leo et al. [29].

graph *in memory*. The Fat Tree uses an Adaptive Radix Tree as the primary index. Its leaf nodes are sparse arrays split into contiguous page-sized segments. The segments store inline the vertices and edges sorted lexicographically, leaving gaps for the adjacency lists to grow. The new elements are inserted into the segment in a sorted manner. Once full, the segment is marked as Write-optimized, and new writes are appended to an unsorted buffer. Eventually, a service thread is triggered to sort the buffer and merge it with the sorted segment. Read-optimized segments are read-friendly, and the Write-optimized segments are used when the burst of writes is high. Teseo performs some optimizations, such as increasing the size of the leaf nodes to several MBs to reduce the number of random index lookups and using a hash table to store pointers to positions in the leaf nodes where the adjacency list for a vertex is stored.

2.4 Graph Processing Systems

We examined ~~the~~ whole graph analytics tasks in §2.2 and now examine the systems designed to run these tasks. **TLS** ▶ *Whether static or dynamic, a graph might be too large to fit in main memory.* ◀ In this scenario, the system designer has two options: distributing computation to multiple machines with enough aggregated RAM to store the graph or running the computation on a single machine, caching part of the graph and any algorithmic state in main memory while keeping the remainder of the graph on persistent storage (e.g., a flash drive). If the graph evolves, the system designer must decide whether and how they ~~would~~ support incremental computing on the graph. We briefly examine the landscape of graph processing systems research. A more comprehensive classification of these systems can be found in a survey by Heidari et al. [38].

2.4.1 Distributed Graph Processing Systems

TLS ▶ *The onset of distributed graph processing is marked by the groundbreaking Pregel [53] paper, and it set the initial state of the research landscape in graph processing.* ◀ In the years following Pregel, distributed graph processing systems became the dominant paradigm for processing large graphs, and several systems were designed around that paradigm. GraphX [33], Gemini [87], Petuum [81], Distributed GraphLab [48], PowerGraph [32], PowerLyr [25], ~~etc.~~ are popular examples of distributed graph processing systems. Distributed graph processing allows ~~graphs to be processed that cannot~~ fit in ~~any~~ one machine's main memory. Several optimizations have been proposed to make distributed graph computation more efficient, including batching **updates**, careful handling of **high-degree vertices** ~~in a graph~~, better partitioning **strategies**, and several others that are summarized in the surveys [38, 55].

TLS ▶ *Using a large cluster for graph processing increases the total amount of memory available and processing power. However, it also introduces communication overhead, which reduces per-core performance [17, Figure 1.1].* ◀ The increased

communication overhead partly explains the shocking results reported by McSherry et al., which demonstrated that an optimized single-threaded implementation of the widely-used Page Rank algorithm outperformed many popular distributed systems using hundreds of cores [56]. This paper underscored the need for better single-node baselines that use performant algorithms to compare distributed graph processing systems.

2.4.2 Shared Memory Graph Processing Systems

TLS ▶ Compared to distributed clusters, shared-memory multiprocessors offer scalability with much lower communication overhead, typically achieving significantly higher per-core performance [17, Section 1.1]. ◀ Thus, shared-memory graph processing systems often perform orders of magnitude faster than their distributed competitors [16, 46, 60, 70, 76, 87]. **PM** ▶ I feel like I should write more about these systems, but I don't know what ◀

2.4.3 Semi-External Graph Processing Systems

Out-of-core systems are designed to keep all or some parts of the graph on a slow(er) storage device such as an SSD or hard drive. An out-of-core system is called **fully external** if the RAM available to the graph processing system is severely limited and not large enough even to store the entire vertex state. On the other hand, **Semi-external** systems have a bigger memory budget and can store the entire vertex state in memory, but the memory is not large enough for all the edges. The design of semi-external systems makes use of the fact that the number of vertices in graphs is an order of magnitude fewer than the number of edges. Therefore, the vertex state can be accommodated in the main memory of a modest workstation. Systems such as CLIP [11], FlashGraph [85], GraphMP [75], TurboGraph [37] are expressly designed to be semi-external, while systems such as GraphChi [45] and X-Stream [65] have an optional semi-external mode (depending on the memory budget).

TLS ▶ DRAM is an expensive resource and can account for roughly 30% of the cost of a modern datacenter [79, Figure 1]. ◀ With the unrelenting growth of graph dataset sizes, keeping all the graph data in memory is increasingly untenable. Fortunately, fast solid-state devices, such as NVMe devices, have become widely available, and they present another architectural opportunity for ~~graph processing~~: semi-external graph processing systems. In this architecture, a small subset of the graph and some algorithmic state is maintained in memory (tunable based on the memory budget); the rest of the graph is streamed from slower external storage. Popular systems that are built with this architecture include GraphChi [45], X-Stream [65], GridGraph [86], FlashGraph [85], Blaze [43], Wonderland [84], Mosaic [50], Garaph [49], GraFBoost [41], and many more. These systems have democratized running graph analytics, allowing users to run computations on their trillion-edged graphs on a single machine at a fraction of the cost of doing the same on a cluster.

2.4.4 Dynamic Graph Processing Systems

TLS ▶ Dynamic graphs are graphs that can evolve, and the graph processing system must be able to handle these updates. ◀ There are two broad classes of dynamic graph processing systems:

(1) **Streaming graph analytics systems**: maintain a single version of the evolving graph, and uses incremental algorithms to update the query results as the graph is modified. Using incremental algorithms avoids recomputing the query from scratch, and the cost of streaming the updates is amortized by using appropriately sized batches. Systems designed for this class of dynamic graph processing include Kineograph [26], Stinger [31], Kickstarter [78], Tornado [69], JetStream [64], GraphBolt [54], Tripoline [40], and GraphOne [44].

(2) **Evolving graph analytics systems**: evaluate a query on multiple snapshots of a graph, that can stretch over a large time window to extract trend data or compute graph properties at discrete time points. Examples of such

systems include Chronos [36], GraphTau [39], LLAMA [51], and ImmortalGraph [58]. The system must be able to efficiently store and retrieve snapshots of the graph and support efficient queries over these snapshots. These systems typically use some optimizations for amortizing the fetch cost by merging fetching operations across snapshots and using the values computed in earlier snapshots to speed up the computation in later snapshots [77]. Another optimization is to execute the query on multiple snapshots in parallel and merge the results to get the final result; alternatively, the system can execute the query on one snapshot at a time and use incremental algorithms to propagate the results to the following snapshot.

2.5 Persistent Graph Layouts

TLS ▶ *So far, we have only examined in-memory data structures used to store graphs.*◀ In the best-case scenario, the graph is small enough to fit entirely in a system’s main memory. However, graphs can often be massive, exceeding the RAM capacity of a single machine. While a potential solution, distributed graph processing is often inefficient due to communication overheads [17, 56]. Additionally, RAM is expensive, making minimizing memory usage in graph processing applications critical. When a graph does not fit in RAM, a persistent (on-disk) graph data layout that allows fast sequential neighborhood scans becomes essential.

TLS ▶ *The in-memory data structures we have examined so far cannot be naively made persistent.*◀ To illustrate, consider a persistent adjacency list design. A persistent adjacency list representation might start by chunking nodes and their incoming/outgoing edges. Each chunk would internally need metadata to identify where a given node’s neighborhood begins. When running graph queries, this representation would require additional metadata to find and read the correct chunk into RAM efficiently. If the graph is updated, the on-disk chunks might need resizing and offset/chunk metadata updates; any cached metadata also needs to be kept consistent with the on-disk metadata, requiring additional I/O operations. The increased I/O for data structure management and metadata consistency makes out-of-core adjacency lists inefficient.

TLS ▶ *An array-based Compressed Sparse Row (CSR) format can be naively persisted using memory-mapped files to store the vertex and index arrays.*◀ However, there are many reasons why using `mmap` for databases is not advisable [27], and similar issues apply to graph processing systems. The operating system can swap out pages, causing unpredictable latency spikes during neighborhood scans. The CSR design is not easily chunked into blocks/pages because the index array and the vertex array are interdependent and cannot be easily partitioned. LLAMA [51] is an example of a persistent (mutable) CSR representation. Its design demonstrates the careful considerations necessary to achieve persistence, balancing the need for efficient data access and updates with the constraints of out-of-core storage.

TLS ▶ *Fortunately, there has been a lot of research into creating on-disk graph data layouts that provide fast sequential access to vertex neighborhoods.*◀ The persistent data layouts depend on whether the system has been designed for static or dynamic graphs, whether the system is fully out-of-core or keeps some algorithmic state or metadata in RAM, and whether the system supports property graphs.

We first examine the persistent representations that static graph processing systems use that keep some state in RAM (semi-external systems) or no state in RAM (out-of-core systems). Then, we examine systems that store evolving graphs; this is quite close to the design point of Flexograph and will be referenced again when describing our evaluation strategy in Chapter 3. We close the discussion of persistent data layouts by examining the cost of preprocessing the graphs to construct these data layouts and whether these costs can be amortized.

2.5.1 Persistent Data Layouts for Static GPS

The persistent layout is designed to perform efficient neighborhood scans for vertices. If the system is fully external, the layout must also consider the efficiency of vertex state updates and propagation to the disk-resident structures.

Each GPS is unique in its layout design and update propagation strategies, so instead of examining several systems individually, we examine a generalized architecture of a fully external graph processing system. **PM** ▶ *this is adapted from a diagram in "Seraph: Towards Scalable and Efficient Fully-external Graph Computation via On-demand Processing."* @MIS: should I redraw this, or is this fine? ◀

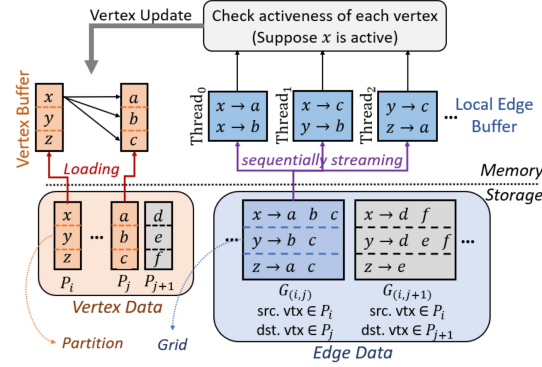


Figure 2.6: A generalized architecture of a fully external graph processing system. Taken from Yang et al. [82].

A fully external GPS essentially shards the graph into multiple subgraphs and stores them on disk, accessing them one at a time for processing. The vertex state, ~~on the other hand,~~ might be stored in memory depending on the system's memory budget, but in the most generalized case, it is also stored on disk. The vertex set is partitioned into disjoint chunks; when processing the edges between two such vertex chunks, the system only needs to store them in memory and stream the edges between them from disk. Out-of-core graph processing systems have different ways to create subgraphs to optimize sequential access to the graph. With this architecture in mind, we now examine the persistent data layouts used by static graph processing systems.

GraphChi stores the subgraphs in shards created such that a shard contains all edges $e = (u, v)$ where v is in the vertex chunk being processed. Within a shard, the edges are sorted by the source vertex ID. The execution proceeds in "execution intervals"; a vertex chunk (called a memory shard) is loaded into memory, and the edges between the chunk and the rest of the graph are streamed from the disk. The shard format is relatively straightforward: it is a binary file of adjacency lists, with each vertex's adjacency list stored as a variable-sized length word (degree) followed by the list of neighbors. Notably, the edge data (weights and updates from computations) are stored separately from the graph's structure. The edge data is mutable in GraphChi and gets completely overwritten several times during the execution of a graph algorithm.

X-Stream [65] takes a different approach to maximize sequential access to the edge data. Unlike GraphChi, X-Stream streams the edges and updates the vertex states in memory, ~~instead~~ relying on the ~~sheer~~ parallelism of ~~using RAID groups~~. X-Stream uses the notion of a *streaming partition*, which is a subset of vertices and all the edges with a *source vertex* in that **subset**. The update list of a streaming partition consists of all updates whose destination vertex is in the partition's vertex set. The graph is partitioned by the source vertex of the edges. Unlike GraphChi, X-Stream does not require a strict ordering of edges within a partition. The premise is that partitioning a graph by the source vertex ID is less computationally expensive than sorting the edges, as GraphChi does. This is why X-Stream is said to have no preprocessing step.

At runtime, the graph is processed by reading the edges of a streaming partition and keeping the vertex state in memory. By default, X-Stream keeps only a subset of the vertex chunk in memory; however, if the vertex set is small enough, the entire vertex set can be kept in memory. Similarly, if the updates from the scatter phase fit in memory, they are not synced to the disk but read from memory for the next gather phase.

GridGraph [86] is a system that uses a grid-based graph partitioning to optimize the sequential access to the graph. As in other systems, the vertex set is partitioned into disjoint chunks, and the edges form a $P \times P$ grid (1-D partitioning). The edges are split into the grid based on the source and destination vertex IDs. The edge (u, v) is placed in the cell $(u \bmod P, v \bmod P)$, i.e. the source determines the row and the destination determines the column (2D partitioning). GridGraph is a fully external GPS that stores the vertex and edge state on disk. The vertex state is stored in a file-backed memory-mapped region, and each edge block is stored in a separate file. The edges in any block need not be sorted. Metadata tracks the global state, such as the number of vertices and edges, the number of partitions, and the edge offsets. GridGraph can reduce the number of I/O operations by using selective streaming, i.e., only streaming edges for "active" vertices; this is aided by fine-grained 2D edge partitioning. The preprocessing step in GridGraph reads the whole graph (pre-converted to a binary format) into memory; the binary format reduces the memory needed and time taken to read the graph from disk. In comparison, GraphChi reads the graph one edge at a time, taking much less memory but more time to read the graph. This grid layout is also used in Blaze [43], Graphene [47], FlashGraph [85], and others. **PM** *I can, if needed, add more systems here. Perhaps when I know which systems I am comparing against.*

2.5.2 Persistent Data Layouts for Dynamic GPS

Several dynamic graph processing systems also use persistent graph data layouts to store evolving graphs. These systems are designed to handle the graph structure and properties changing over time, storing them such that the graph can be queried at any point in its evolution. Since the graph is now mutable, the data structures must incorporate some form of versioning and transactional support. There are many ways to design a persistent data layout for evolving graphs, and we examine data layouts used by two systems: Teseo [?] and LiveGraph [88].

Transactional Edge Log **TLS** *LiveGraph [88] is a recent graph storage system that allows purely sequential access to a graph's adjacency list.* The graph is stored in memory-mapped files, and the adjacency list is stored in a file-backed memory-mapped region. The vertices are arranged in vertex blocks; they contain the vertex properties and a pointer to the previous vertex version. A vertex index is used to look up the most recent version of a vertex's vertex block; past versions can be found by following the version chain. The edges are stored in edge blocks, each containing the adjacency list for one source vertex; an index is used to look up the edge block for a source vertex. The edge blocks have two parts - the edge log and the property entries. The edge properties are stored in a separate part of the edge block and contain the variable size properties of each edge. The adjacency lists are stored as log entries in the edge block. Each entry contains the destination vertex ID, creation, invalidation timestamps, and the size of the property entry. During a sequential read, the invalidation timestamps determine whether the edge is still valid (and should be considered). The log-structured design makes inserting new edges expensive since the previous version's invalidation timestamp needs to be set for each new edge. To alleviate this cost, LiveGraph uses a bloom filter in the Edge Block to quickly determine if an edge is present and, therefore, whether the invalidation timestamp needs to be updated. The edge blocks are versioned, and pointers in the edge block link versions. **PM** *I can add the diagram, but I don't think it is necessary.*

This design has many advantages: the adjacency list is stored sequentially, the property blocks are stored separately, and the versioning allows for fast lookups of past versions of the graph. However, using memory-mapped files for graph storage is likely a wrong design choice, as discussed in [27]. The use of memory-mapped files can lead to unpredictable latency spikes due to the operating system swapping out pages, further complicating the design

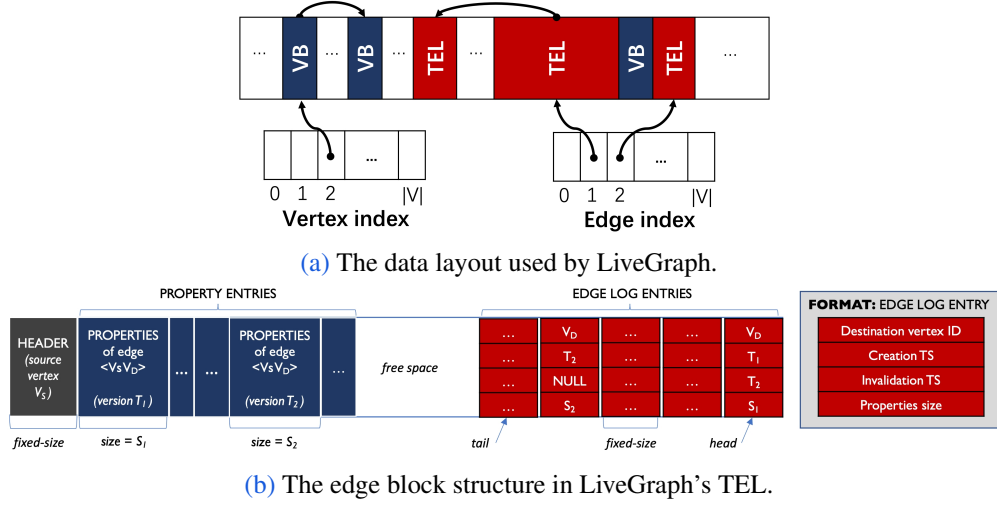


Figure 2.7: The data layout used by LiveGraph. Figure 2.7a shows the high-level layout used by LiveGraph, indicating the indexes used to find vertex and edge blocks in LiveGraph’s TEL. The edge block structure in Figure 2.7b shows the format of an edge log entry and how the adjacency list is stored separately from the property blocks. Taken from Zhu et al. [88].

of the transaction protocol.

2.6 Graph Databases

Databases are the primary persistent store for most data, and graphs are no exception. The development of graph databases began with retrofitting relational databases to work with graph data, and this approach is reasonable but not optimal. The graph database landscape can be classified using the storage backends that the system uses, the data organization strategies it uses, or the query language and graph models it uses to store and query the data. Besta et al. [18] provides a comprehensive overview of the graph database landscape, and we encourage the reader to refer to it for a more detailed discussion. We describe briefly the data layouts and optimizations common in graph databases and how these are used in three commercial graph databases: Neo4j, ArangoDB, and JanusGraph.

2.6.1 Data Layouts used by Graph Databases

A graph database can store graphs using a general-purpose storage engine or a custom storage engine and layout. Common general-purpose storage engines used by graph databases include key-value stores, column-family stores, and document stores. Each of these storage engines has its own data layout, and the graph database designer must instead focus on how to store the graph element in the layout provided by these storage engines.

A detailed discussion of these storage engines is beyond the scope of this thesis, but we provide a brief overview of the data layouts used by the storage engines used in Neo4j, ArangoDB, and JanusGraph. A more detailed discussion of these general-purpose NoSQL storage engines can be found in the survey by Davoudian et al. [28], and a detailed discussion of the other storage engines used by graph databases can be found in the survey by Besta et al. [18].

Neo4j is the most popular graph database system, and it follows the property graph model. Neo4j uses a custom storage engine that uses fixed-size records to store the graph elements. The data layout used by Neo4j is shown in

Figure 2.8. Vertices are stored in vertex records, and edges are stored in **edge records**. A vertex record contains the vertex ID, the labels associated with that vertex, the pointer to the first edge adjacent to the vertex, and some metadata flags. An Edge record contains the edge label, a pointer to the linked list of the edge properties, pointers to the source and destination vertex records, pointers to the adjacency lists of the source and destination vertices, and some metadata flags. Adjacency lists are stored as **doubly linked lists**; each edge is part of two adjacency lists, one for the source vertex and one for the destination vertex, and therefore has two previous pointers and two next pointers.

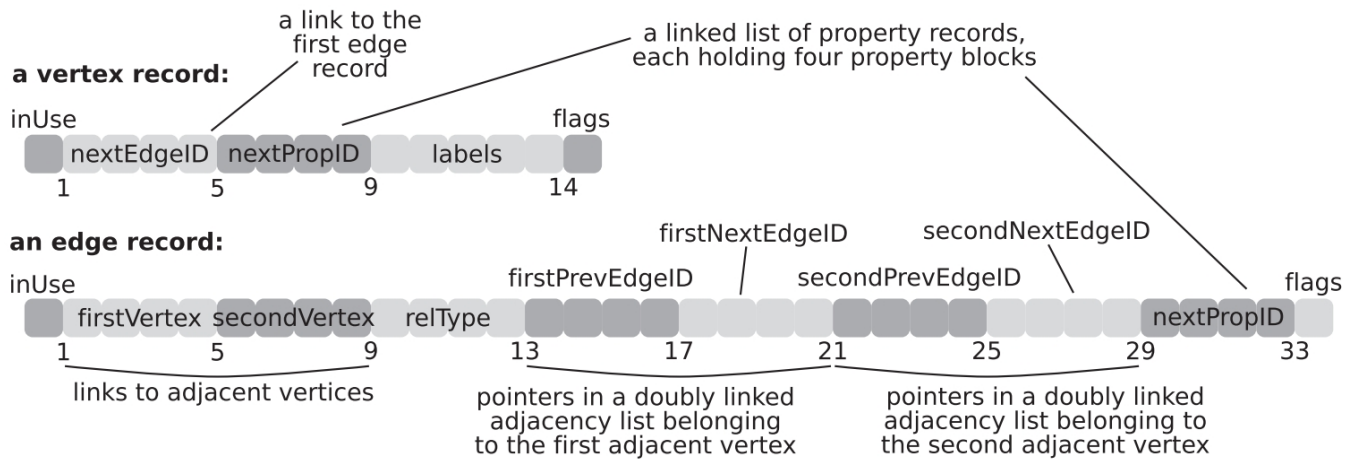


Figure 2.8: An overview of the Neo4j vertex and edge records. Taken from Besta et al. [18].

Storing the edge and vertex properties separately from the edge and vertex records allows for small record sizes, which increases access efficiency since the properties are accessed only if the query needs to. Neo4j is unique in its use of direct pointers to store the locations of a vertex's neighbors, which is the foundation of its claim of **"index-free adjacency."** A traversal can be done without the need to look up any index to find the neighbors of a vertex, and this is significant since the query complexity is no longer linked to the size of the graph. However, due to this overreliance on direct pointers, Neo4j **cannot scale to graphs too large to fit in memory**. Moreover, query efficiency depends entirely on the system's caching and cache pre-warming strategies, which can lead to unpredictable runtime behavior.

ArangoDB is another open-source document store multi-model database that uses RocksDB as its storage engine. Document stores organize **documents into collections**, and each document is a JSON object; each document in a collection has a unique key, `_key`, that stores the ID of the graph element being stored in that document.

ArangoDB uses two collections to represent graphs: one for the vertices and one for the edges. Each collection has a primary index on the `_key` field, and the `_key` field is used to store the vertex ID or the edge ID. The vertex collection stores the vertex properties but not the adjacency list. This design is helpful for fast traversals and transactions that alter the edges but not the vertex properties. The edge collection stores, for each edge, its properties and the source and destination vertex IDs (in the `_from` and `_to` fields; the collection can have secondary indexes on the `_from` and `_to` fields).

ArangoDB uses an interesting hybrid index on the edge collections that allows finding the neighborhoods of a vertex without needing to look up an index, as shown in Figure 2.9. According to the documentation [14], ArangoDB "stores all edges in a big hash table, and at the same time puts for each vertex V all those edges that are incident with V into a doubly linked list." With this index, seeking the linked list to a vertex's adjacency list is an $O(1)$ operation, and if there are k neighbors, the traversal is $O(k)$. This index is also used to speed up single-edge modifications and deletions. The vertex index is used to find the vertex's adjacency list in the optimistic case, where the sought edge

is the first edge in the adjacency list. If that search fails, the system uses the hash table to find the edge. This design allows for $O(1)$ access for single-edge updates.

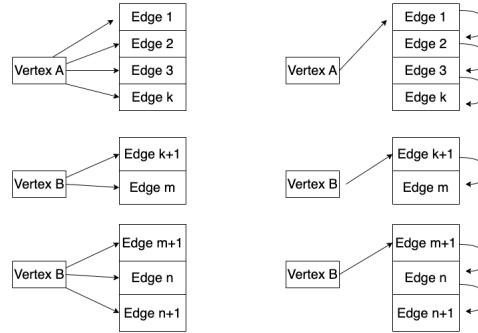


Figure 2.9: The hybrid index used by ArangoDB to achieve index-free adjacency. The edges are stored in a hash table, and the edges to the same vertex are linked together, allowing better traversals. Reproduced with modifications from the ArangoDB documentation [14]. **PM** ▶ make this diagram more intuitive ◀

JanusGraph [7] is a distributed graph database that uses wide-column stores such as **Apache Cassandra** or **Apache HBase** as its storage engine. The data layout is defined in terms of rows and cells in the wide-column store, as shown in Figure 2.10. Regardless of the choice of the storage engine, each row in the wide-column store represents a vertex. The vertex properties are stored in the row, followed by the adjacent edges; each property and edge are stored in a separate cell. The rows in a wide-column store are sorted by the row key (the vertex ID), and the cell key sorts the cells in a row. The cell keys for the properties and edges are chosen so that the properties are stored before the edges. The graph can be partitioned across multiple nodes in the cluster, and the partitioning is done based on the vertex ID.

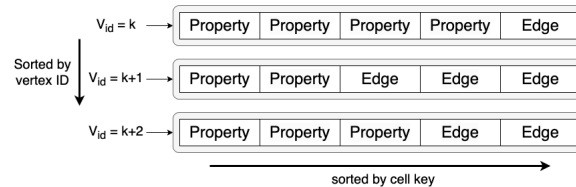


Figure 2.10: The data layout used by JanusGraph to store graphs in a wide-column store. The vertex properties and adjacent edges are stored in the row, each in their cells. The cells are sorted by the cell key, and the rows are sorted by the vertex ID. Taken from [18].

Chapter 3

Flexograph Architecture

3.1 Representations

3.2 GraphAPI

3.3 Iterator Design

3.4 Semi-External Processing

3.5 Application Design

3.6 Evaluation

Chapter 4

Storing Property Graphs

Chapter 5

Specialized Indexes on Graph Data

Chapter 6

Future Work

Chapter 7

timeline

Bibliography

- [1] Ldbc introduction. <https://ldbcouncil.org/introduction/>. (Accessed on 05/18/2024). → page 8
- [2] Ldbc graphalytics benchmark (ldbc graphalytics). <https://ldbcouncil.org/benchmarks/graphalytics/>, . (Accessed on 05/18/2024). → page 8
- [3] Ldbc social network benchmark (ldbc snb). <https://ldbcouncil.org/benchmarks/snb/>, . (Accessed on 05/18/2024). → page 8
- [4] Wiredtiger overview and features. <http://source.wiredtiger.com/10.0.0/overview.html>. → page 5
- [5] Arangodb: Open-source multi-model database for graph, document and search. <https://www.arangodb.com/>. → pages 1, 7
- [6] Graph500 benchmarks. <https://graph500.org/>. → page 3
- [7] Janusgraph: Distributed, open source, massively scalable graph database. <https://janusgraph.org/>. → page 19
- [8] Neo4j. <https://neo4j.com/>. → pages 1, 4, 7
- [9] Vertica. <https://www.vertica.com>. → page 1
- [10] C. R. Aberger, A. Lamb, S. Tu, A. Nötzli, K. Olukotun, and C. Ré. Emptyheaded: A relational engine for graph processing. *ACM Trans. Database Syst.*, 42(4), oct 2017. ISSN 0362-5915. doi:10.1145/3129246. URL <https://doi.org/10.1145/3129246>. → page 1
- [11] Z. Ai, M. Zhang, Y. Wu, X. Qian, K. Chen, and W. Zheng. Clip: A disk i/o focused parallel out-of-core graph processing system. *IEEE Transactions on Parallel and Distributed Systems*, 30(1):45–62, 2018. → page 13
- [12] R. Angles. The property graph database model. In *AMW*, 2018. → page 5
- [13] R. Angles, M. Arenas, P. Barceló, A. Hogan, J. Reutter, and D. Vrgoč. Foundations of modern query languages for graph databases. *ACM Computing Surveys*, 50(5):1–40, Sept. 2018. ISSN 0360-0300, 1557-7341. doi:10.1145/3104031. → page 7
- [14] ArangoDB. Index free adjacency or hybrid indexes for graph databases. <https://arangodb.com/2016/04/index-free-adjacency-hybrid-indexes-graph-databases/>, April 2016. (Accessed on 05/31/2024). → pages 18, 19

- [15] J. Arulraj, A. Pavlo, and P. Menon. Bridging the archipelago between row-stores and column-stores for hybrid workloads. In *Proceedings of the 2016 International Conference on Management of Data, SIGMOD '16*, page 583–598, New York, NY, USA, 2016. Association for Computing Machinery. ISBN 9781450335317. doi:10.1145/2882903.2915231. URL <https://doi.org/10.1145/2882903.2915231>. → page 4
- [16] S. Beamer, K. Asanović, and D. Patterson. The gap benchmark suite. *arXiv preprint arXiv:1508.03619*, 2015. → pages 3, 13
- [17] S. Beamer III. *Understanding and improving graph algorithm performance*. University of California, Berkeley, 2016. → pages 2, 12, 13, 14
- [18] M. Besta, R. Gerstenberger, E. Peter, M. Fischer, M. Podstawski, C. Barthels, G. Alonso, and T. Hoefler. Demystifying graph databases: Analysis and taxonomy of data organization, system designs, and graph queries. *ACM Comput. Surv.*, 56(2), sep 2023. ISSN 0360-0300. doi:10.1145/3604932. URL <https://doi.org/10.1145/3604932>. → pages 1, 7, 8, 17, 18, 19
- [19] J. Biskup and H. Stiefeling. Transitive closure algorithms for very large databases. In J. van Leeuwen, editor, *Graph-theoretic concepts in computer science*, page 122–147, Berlin, Heidelberg, 1989. Springer Berlin Heidelberg. ISBN 978-3-540-46076-3. → page 1
- [20] P. A. Boncz, M. Zukowski, and N. Nes. Monetdb/x100: Hyper-pipelining query execution. In *Cidr*, volume 5, pages 225–237. Citeseer, 2005. → page 1
- [21] A. Bonifati, G. Fletcher, H. Voigt, N. Yakovets, and H. Jagadish. *Querying graphs*, volume 10. Springer, 2018. → page 7
- [22] N. Bronson, Z. Amsden, G. Cabrera, P. Chakka, P. Dimov, H. Ding, J. Ferris, A. Giardullo, S. Kulkarni, H. Li, et al. TAO: Facebook’s distributed data store for the social graph. In *2013 USENIX Annual Technical Conference USENIX ATC 13*, pages 49–60, 2013. → page 1
- [23] M. Capotă, T. Hegeman, A. Iosup, A. Prat-Pérez, O. Erling, and P. Boncz. Graphalytics: A big data benchmark for graph-processing platforms. In *Proceedings of the GRADES’15*, page 7. ACM, 2015. → page 3
- [24] D. Chakrabarti and C. Faloutsos. Graph mining: Laws, generators, and algorithms. *ACM Comput. Surv.*, 38(1):2–es, jun 2006. ISSN 0360-0300. doi:10.1145/1132952.1132954. URL <https://doi.org/10.1145/1132952.1132954>. → page 9
- [25] R. Chen, J. Shi, Y. Chen, B. Zang, H. Guan, and H. Chen. Powerlyra: Differentiated graph computation and partitioning on skewed graphs. *ACM Trans. Parallel Comput.*, 5(3), jan 2019. ISSN 2329-4949. doi:10.1145/3298989. URL <https://doi.org/10.1145/3298989>. → pages 1, 12
- [26] R. Cheng, J. Hong, A. Kyrola, Y. Miao, X. Weng, M. Wu, F. Yang, L. Zhou, F. Zhao, and E. Chen. Kineograph: taking the pulse of a fast-changing and connected world. In *Proceedings of the 7th ACM european conference on Computer Systems*, pages 85–98, 2012. → page 13
- [27] A. Crotty, V. Leis, and A. Pavlo. Are you sure you want to use mmap in your database management system. In *CIDR 2022, Conference on Innovative Data Systems Research*. <https://db.cs.cmu.edu/papers/2022/p13-crotty.pdf>, 2022. → pages 14, 16

- [28] A. Davoudian, L. Chen, and M. Liu. A survey on nosql stores. *ACM Comput. Surv.*, 51(2), apr 2018. ISSN 0360-0300. doi:[10.1145/3158661](https://doi.org/10.1145/3158661). URL <https://doi.org/10.1145/3158661>. → page 17
- [29] D. De Leo and P. Boncz. Teseo and the analysis of structural dynamic graphs. *Proceedings of the VLDB Endowment*, 14(6):1053–1066, 2021. → pages 11, 12
- [30] R. Diestel and R. Diestel. Graph theory. *Graph Theory*, pages 1–34, 2017. → page 6
- [31] D. Ediger, R. McColl, J. Riedy, and D. A. Bader. Stinger: High performance data structure for streaming graphs. In *2012 IEEE Conference on High Performance Extreme Computing*, pages 1–5. IEEE, 2012. → page 13
- [32] J. E. Gonzalez, Y. Low, H. Gu, D. Bickson, and C. Guestrin. Powergraph: Distributed graph-parallel computation on natural graphs. In *Presented as part of the 10th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 12)*, pages 17–30, 2012. → page 12
- [33] J. E. Gonzalez, R. S. Xin, A. Dave, D. Crankshaw, M. J. Franklin, and I. Stoica. Graphx: Graph processing in a distributed dataflow framework. In *OSDI*, volume 14, pages 599–613, 2014. → pages 1, 12
- [34] A. Gupta, D. Agarwal, D. Tan, J. Kulesza, R. Pathak, S. Stefani, and V. Srinivasan. Amazon redshift and the case for simpler data warehouses. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, page 1917–1923, Melbourne Victoria Australia, May 2015. ACM. ISBN 978-1-4503-2758-9. doi:[10.1145/2723372.2742795](https://doi.org/10.1145/2723372.2742795). URL <https://dl.acm.org/doi/10.1145/2723372.2742795>. tex.ids= guptaAmazonRedshiftCase2015. → page 1
- [35] P. Gupta, A. Mhedhbi, and S. Salihoglu. Columnar storage and list-based processing for graph database management systems. *Proc. VLDB Endow.*, 14(11):2491–2504, jul 2021. ISSN 2150-8097. doi:[10.14778/3476249.3476297](https://doi.org/10.14778/3476249.3476297). URL <https://doi.org/10.14778/3476249.3476297>. → page 4
- [36] W. Han, Y. Miao, K. Li, M. Wu, F. Yang, L. Zhou, V. Prabhakaran, W. Chen, and E. Chen. Chronos: a graph engine for temporal graph analysis. In *Proceedings of the Ninth European Conference on Computer Systems*, pages 1–14, 2014. → page 14
- [37] W.-S. Han, S. Lee, K. Park, J.-H. Lee, M.-S. Kim, J. Kim, and H. Yu. Turbograph: a fast parallel graph engine handling billion-scale graphs in a single pc. In *Proceedings of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 77–85, 2013. → page 13
- [38] S. Heidari, Y. Simmhan, R. N. Calheiros, and R. Buyya. Scalable graph processing frameworks: A taxonomy and open challenges. *ACM Comput. Surv.*, 51(3):60:1–60:53, June 2018. ISSN 0360-0300. doi:[10.1145/3199523](https://doi.org/10.1145/3199523). URL <http://doi.acm.org/10.1145/3199523>. → page 12
- [39] A. P. Iyer, L. E. Li, T. Das, and I. Stoica. Time-evolving graph processing at scale. In *Proceedings of the fourth international workshop on graph data management experiences and systems*, pages 1–6, 2016. → page 14
- [40] X. Jiang, C. Xu, X. Yin, Z. Zhao, and R. Gupta. Tripoline: generalized incremental graph processing via graph triangle inequality. In *Proceedings of the Sixteenth European Conference on Computer Systems*, pages 17–32, 2021. → page 13

- [41] S.-W. Jun, A. Wright, S. Zhang, S. Xu, et al. Grafboost: Using accelerated flash storage for external graph analytics. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, pages 411–424. IEEE, 2018. → page 13
- [42] R. Kallman, H. Kimura, J. Natkins, A. Pavlo, A. Rasin, S. Zdonik, E. P. C. Jones, S. Madden, M. Stonebraker, Y. Zhang, J. Hugg, and D. J. Abadi. H-store: a high-performance, distributed main memory transaction processing system. *Proc. VLDB Endow.*, 1(2):1496–1499, aug 2008. ISSN 2150-8097. doi:10.14778/1454159.1454211. URL <https://doi.org/10.14778/1454159.1454211>. → page 1
- [43] J. Kim and S. Swanson. Blaze: fast graph processing on fast ssds. In *2022 SC22: International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, pages 617–631. IEEE Computer Society, 2022. → pages 13, 16
- [44] P. Kumar and H. H. Huang. Graphone: A data store for real-time analytics on evolving graphs. In *17th USENIX Conference on File and Storage Technologies (FAST 19)*, pages 249–263, Boston, MA, Feb. 2019. USENIX Association. ISBN 978-1-939133-09-0. URL <https://www.usenix.org/conference/fast19/presentation/kumar>. → page 13
- [45] A. Kyrola, G. E. Blelloch, and C. Guestrin. Graphchi: Large-scale graph computation on just a pc. In *OSDI. USENIX*, 2012. → pages 1, 13
- [46] K. Lakhotia, R. Kannan, S. Pati, and V. Prasanna. Gpop: A scalable cache-and memory-efficient framework for graph processing over parts. *ACM Transactions on Parallel Computing (TOPC)*, 7(1):1–24, 2020. → page 13
- [47] H. Liu and H. H. Huang. Graphene: Fine-Grained IO management for graph computing. In *15th USENIX Conference on File and Storage Technologies (FAST 17)*, pages 285–300, Santa Clara, CA, Feb. 2017. USENIX Association. ISBN 978-1-931971-36-2. URL <https://www.usenix.org/conference/fast17/technical-sessions/presentation/liu>. → page 16
- [48] Y. Low, D. Bickson, J. Gonzalez, C. Guestrin, A. Kyrola, and J. M. Hellerstein. Distributed graphlab: a framework for machine learning and data mining in the cloud. *Proceedings of the VLDB Endowment*, 5(8): 716–727, 2012. → page 12
- [49] L. Ma, Z. Yang, H. Chen, J. Xue, and Y. Dai. Garaph: Efficient {GPU-accelerated} graph processing on a single machine with balanced replication. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)*, pages 195–207, 2017. → page 13
- [50] S. Maass, C. Min, S. Kashyap, W. Kang, M. Kumar, and T. Kim. Mosaic: Processing a trillion-edge graph on a single machine. In *Proceedings of the Twelfth European Conference on Computer Systems*, pages 527–543, 2017. → page 13
- [51] P. Macko, V. J. Marathe, D. W. Margo, and M. I. Seltzer. LLAMA: Efficient graph analytics using Large Multiversioned Arrays. In *2015 IEEE 31st International Conference on Data Engineering*, 2015. → pages 11, 14
- [52] K. Madduri and D. A. Bader. Compact graph representations and parallel connectivity algorithms for massive dynamic network analysis. In *2009 IEEE International Symposium on Parallel & Distributed Processing*, pages 1–11. IEEE, 2009. → page 10

- [53] G. Malewicz, M. H. Austern, A. J. Bik, J. C. Dehnert, I. Horn, N. Leiser, and G. Czajkowski. Pregel: a system for large-scale graph processing. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*, pages 135–146. ACM, 2010. → pages 1, 12
- [54] M. Mariappan and K. Vora. Graphbolt: Dependency-driven synchronous processing of streaming graphs. In *Proceedings of the Fourteenth EuroSys Conference 2019*, pages 1–16, 2019. → page 13
- [55] R. R. McCune, T. Weninger, and G. Madey. Thinking like a vertex: A survey of vertex-centric frameworks for large-scale distributed graph processing. *ACM Comput. Surv.*, 48(2), oct 2015. ISSN 0360-0300. doi:10.1145/2818185. URL <https://doi.org/10.1145/2818185>. → page 12
- [56] F. McSherry, M. Isard, and D. G. Murray. Scalability! but at what cost? In *HotOS*, volume 15, pages 14–14. Citeseer, 2015. → pages 2, 3, 13, 14
- [57] P. Mehrotra, V. Anand, D. Margo, M. R. Hajidehi, and M. Seltzer. Sok: The faults in our graph benchmarks, 2024. → pages 2, 3
- [58] Y. Miao, W. Han, K. Li, M. Wu, F. Yang, L. Zhou, V. Prabhakaran, E. Chen, and W. Chen. Immortalgraph: A system for storage and analysis of temporal graphs. *ACM Trans. Storage*, 11(3), jul 2015. ISSN 1553-3077. doi:10.1145/2700302. URL <https://doi.org/10.1145/2700302>. → page 14
- [59] Neo4j. Understanding neo4j’s data on disk - knowledge base. <https://neo4j.com/developer/kb/understanding-data-on-disk/>. (Accessed on 04/16/2024). → page 4
- [60] D. Nguyen, A. Lenharth, and K. Pingali. A lightweight infrastructure for graph analytics. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, pages 456–471. ACM, 2013. → page 13
- [61] A. Pacaci, A. Zhou, J. Lin, and M. T. Özsu. Do we need specialized graph databases? benchmarking real-time social networking applications. GRADES’17, New York, NY, USA, 2017. Association for Computing Machinery. ISBN 9781450350389. doi:10.1145/3078447.3078459. URL <https://doi.org/10.1145/3078447.3078459>. → page 4
- [62] X. Qiu, W. Cen, Z. Qian, Y. Peng, Y. Zhang, X. Lin, and J. Zhou. Real-time constrained cycle detection in large dynamic graphs. *Proceedings of the VLDB Endowment*, 11(12):1876–1888, 2018. this paper talks about the 20ms latency standard for cycle detection tasks in financial fraud detection systems. → page 2
- [63] M. Raasveldt and H. Mühleisen. Duckdb: an embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data*, pages 1981–1984, 2019. → page 1
- [64] S. Rahman, M. Afarin, N. Abu-Ghazaleh, and R. Gupta. Jetstream: Graph analytics on streaming data with event-driven hardware accelerator. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 1091–1105, 2021. → page 13
- [65] A. Roy, I. Mihailovic, and W. Zwaenepoel. X-stream: Edge-centric graph processing using streaming partitions. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, pages 472–488. ACM, 2013. → pages 13, 15
- [66] Y. Saad. *Iterative methods for sparse linear systems*. SIAM, 2003. → page 10

- [67] S. Sahu, A. Mhedhbi, S. Salihoglu, J. Lin, and M. T. Özsu. The ubiquity of large graphs and surprising challenges of graph processing. *Proc. VLDB Endow.*, 11(4):420–431, Dec. 2017. ISSN 2150-8097. doi:10.1145/3164135.3164139. URL <https://doi.org/10.1145/3164135.3164139>. → page 3
- [68] S. Shen, Z. Yao, L. Shi, L. Wang, L. Lai, Q. Tao, L. Su, R. Chen, W. Yu, H. Chen, B. Zang, and J. Zhou. Bridging the gap between relational OLTP and graph-based OLAP. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 181–196, Boston, MA, July 2023. USENIX Association. ISBN 978-1-939133-35-9. URL <https://www.usenix.org/conference/atc23/presentation/shen>. → page 4
- [69] X. Shi, B. Cui, Y. Shao, and Y. Tong. Tornado: A system for real-time iterative analysis over evolving data. In *Proceedings of the 2016 International Conference on Management of Data*, pages 417–430, 2016. → page 13
- [70] J. Shun and G. E. Blelloch. Ligra: a lightweight graph processing framework for shared memory. In *ACM Sigplan Notices*, volume 48, pages 135–146. ACM, 2013. → pages 1, 13
- [71] SNAP. Snap: Network datasets: Epinions social network. <https://snap.stanford.edu/data/soc-Epinions1.html>. (Accessed on 05/01/2024). → page 9
- [72] C. Staudt, A. Sazonovs, and H. Meyerhenke. Networkit: An interactive tool suite for high-performance network analysis. *arXiv preprint arXiv:1403.3005*, 2014. → page 10
- [73] M. Stonebraker and A. Weisberg. The voltdb main memory dbms. *IEEE Data Eng. Bull.*, 36(2):21–27, 2013. → page 1
- [74] M. Stonebraker, D. J. Abadi, A. Batkin, X. Chen, M. Cherniack, M. Ferreira, E. Lau, A. Lin, S. Madden, E. O’Neil, et al. C-store: a column-oriented dbms. In *Making Databases Work: the Pragmatic Wisdom of Michael Stonebraker*, pages 491–518. 2018. → page 1
- [75] P. Sun, Y. Wen, T. N. B. Duong, and X. Xiao. Graphmp: An efficient semi-external-memory big graph processing system on a single machine. In *2017 IEEE 23rd International Conference on Parallel and Distributed Systems (ICPADS)*, pages 276–283. IEEE, 2017. → page 13
- [76] N. Sundaram, N. Satish, M. M. A. Patwary, S. R. Dulloor, M. J. Anderson, S. G. Vadlamudi, D. Das, and P. Dubey. Graphmat: High performance graph analytics made productive. *Proc. VLDB Endow.*, 8(11): 1214–1225, July 2015. ISSN 2150-8097. doi:10.14778/2809974.2809983. URL <https://doi.org/10.14778/2809974.2809983>. → page 13
- [77] K. Vora, R. Gupta, and G. Xu. Synergistic analysis of evolving graphs. *ACM Trans. Archit. Code Optim.*, 13(4), oct 2016. ISSN 1544-3566. doi:10.1145/2992784. URL <https://doi.org/10.1145/2992784>. → page 14
- [78] K. Vora, R. Gupta, and G. Xu. Kickstarter: Fast and accurate computations on streaming graphs via trimmed approximations. In *Proceedings of the twenty-second international conference on architectural support for programming languages and operating systems*, pages 237–251, 2017. → page 13
- [79] J. Weiner, N. Agarwal, D. Schatzberg, L. Yang, H. Wang, B. Sanouillet, B. Sharma, T. Heo, M. Jain, C. Tang, et al. Tmo: Transparent memory offloading in datacenters. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 609–621, 2022. → page 13

- [80] B. Wheatman and H. Xu. Packed compressed sparse row: A dynamic graph representation. In *2018 IEEE High Performance extreme Computing Conference (HPEC)*, pages 1–7, 2018.
doi:[10.1109/HPEC.2018.8547566](https://doi.org/10.1109/HPEC.2018.8547566). → page 10
- [81] E. P. Xing, Q. Ho, W. Dai, J.-K. Kim, J. Wei, S. Lee, X. Zheng, P. Xie, A. Kumar, and Y. Yu. Petuum: A new platform for distributed machine learning on big data. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1335–1344, 2015. → page 12
- [82] T.-Y. Yang, Y. Chen, Y. Liang, and M.-C. Yang. Seraph: Towards scalable and efficient fully-external graph computation via on-demand processing. In *22nd USENIX Conference on File and Storage Technologies (FAST 24)*, pages 373–387, Santa Clara, CA, Feb. 2024. USENIX Association. ISBN 978-1-939133-38-0. URL <https://www.usenix.org/conference/fast24/presentation/yang-tsun-yu>. → page 15
- [83] W. Yang, T. Li, G. Fang, and H. Wei. Pase: Postgresql ultra-high-dimensional approximate nearest neighbor search extension. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, SIGMOD ’20, page 2241–2253, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450367356. doi:[10.1145/3318464.3386131](https://doi.org/10.1145/3318464.3386131). URL <https://doi.org/10.1145/3318464.3386131>. → page 1
- [84] M. Zhang, Y. Wu, Y. Zhuo, X. Qian, C. Huan, and K. Chen. Wonderland: A novel abstraction-based out-of-core graph processing system. *ACM SIGPLAN Notices*, 53(2):608–621, 2018. → page 13
- [85] D. Zheng, D. Mhembere, R. Burns, J. Vogelstein, C. E. Priebe, and A. S. Szalay. {FlashGraph}: Processing {Billion-Node} graphs on an array of commodity {SSDs}. In *13th USENIX Conference on File and Storage Technologies (FAST 15)*, pages 45–58, 2015. → pages 13, 16
- [86] X. Zhu, W. Han, and W. Chen. {GridGraph}:{Large-Scale} graph processing on a single machine using 2-level hierarchical partitioning. In *2015 USENIX Annual Technical Conference (USENIX ATC 15)*, pages 375–386, 2015. → pages 13, 16
- [87] X. Zhu, W. Chen, W. Zheng, and X. Ma. Gemini: A computation-centric distributed graph processing system. In *12th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 16)*, pages 301–316, 2016. → pages 12, 13
- [88] X. Zhu, G. Feng, M. Serafini, X. Ma, J. Yu, L. Xie, A. Aboulmaga, and W. Chen. Livegraph: A transactional graph storage system with purely sequential adjacency list scans. *arXiv preprint arXiv:1910.05773*, 2019. → pages 16, 17